H4: Beslissingen

Labo 2 · week 4 · samengestelde voorwaarden en scope

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Opwarmen**: `if`, `else if` en `else` uit labo 1
2. **Logische operatoren**: `&&`, `||` en `!`
3. **Vereenvoudigen**: minder code dankzij samengestelde voorwaarden
4. **Valkuilen**: intervallen en de volgorde van `&&` en `||`
5. **Scope**: waar bestaat een variabele?

 *Vandaag: `&&` en `||`. Dubbel zoveel tekens, dubbel zoveel plezier.*

Opwarmen

Wat is de output? 🤔

```
 1  int number = 0;
 2  if (number > 0)
 3  {
 4      Console.WriteLine("Positief");
 5  }
 6  else if (number < 0)
 7  {
 8      Console.WriteLine("Negatief");
 9  }
10  else
11  {
12      Console.WriteLine("Nul");
13  }
```

```
Nul
```

Nul is niet positief en niet negatief. Het Zwitserland van de getallen.

Logische operatoren

EN, OF en NIET

C#-syntax	Betekenis	Wanneer `true`?
`&&`	en-operator	als **beide** kanten `true` zijn
`||`	of-operator	als **minstens één** kant `true` is
`!`	niet-operator	draait het resultaat **om**

Je zet `&&` en `||` **tussen** twee booleaanse expressies, en `!` **ervoor**.

 *Wie technologie of elektronica gehad heeft: ja, dat zijn jullie AND-, OR- en NOT-poorten. Zonder soldeerbout deze keer.*

`&&`: allebei

Je mag de rollercoaster op als je **minstens 1,40 m** bent **en minstens 12 jaar**.

```
1  int height = 145;
2  int age = 11;
3  bool mayRide = height >= 140 && age >= 12;
4  Console.WriteLine(mayRide);  // False
```

Groot genoeg, maar te jong: één kant is `false`, dus het geheel is `false`.

Allebei waar, of je blijft aan de kant met de suikerspin.

`||`: minstens één

Je krijgt korting op het openbaar vervoer als je **jonger dan 18 of ouder dan 65** bent.

```
1  int age = 70;
2  if (age < 18 || age > 65)
3  {
4      Console.WriteLine("Korting op het openbaar vervoer!");
5  }
```

`70 < 18` is `false`, maar `70 > 65` is `true`. Eén van de twee volstaat.

😈 `||`: *de enige operator die tevreden is met de helft.*

`!`: omdraaien

```
1  Console.WriteLine(!(0 == 2));  // True
2
3  bool isRaining = false;
4  if (!isRaining)
5  {
6      Console.WriteLine("Laat de paraplu maar thuis.");
7  }
```

Lees `!` hardop als **“niet”**: “als het **niet** regent”.

 *`!true` is `false`. Net als “ik heb alle oefeningen van vorige week gemaakt”.*

Waarheidstabellen

`&&` (EN)

`&&`	`false`	`true`
`false`	`false`	`false`
`true`	`false`	`true`

`||` (OF)

`||`	`false`	`true`
`false`	`false`	`true`
`true`	`true`	`true`

- `&&` is enkel `true` als **alles** `true` is.
- `||` is enkel `false` als **alles** `false` is.



Waarheidstabellen: de enige tabellen waarin nooit gelogen wordt.

Test jezelf 🤔

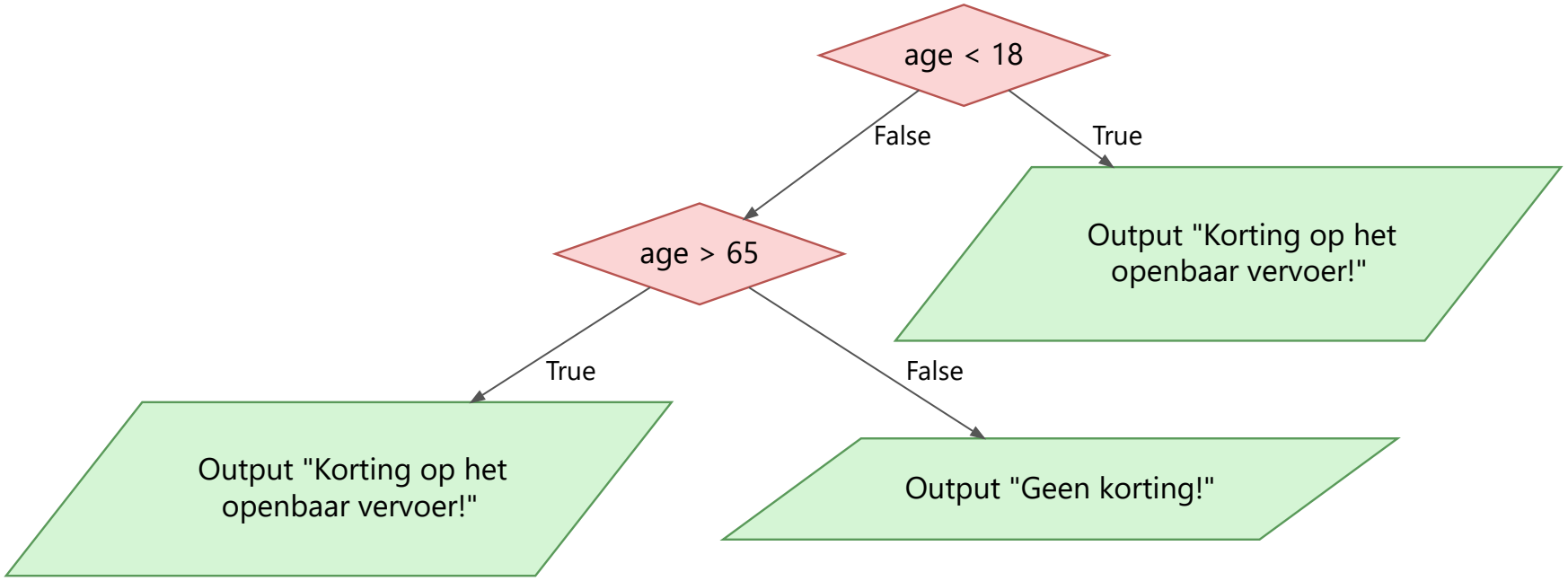
Is het resultaat `true` of `false`?

1. `3 > 2` → `true`
2. `4 != 4` → `false`
3. `4 < 5 && 4 < 3` → `false`
4. `"a" == "a" || 4 >= 3` → `true`
5. `(3 == 3 && 2 < 1) || 5 != 4` → `true`
6. `!(4 <= 3)` → `true`
7. `true || false` → `true`
8. `!true && false` → `false`

Acht vragen, telkens `true` of `false`. Gokken geeft gemiddeld 50%. Net niet geslaagd.

Vereenvoudigen

Zonder `||`



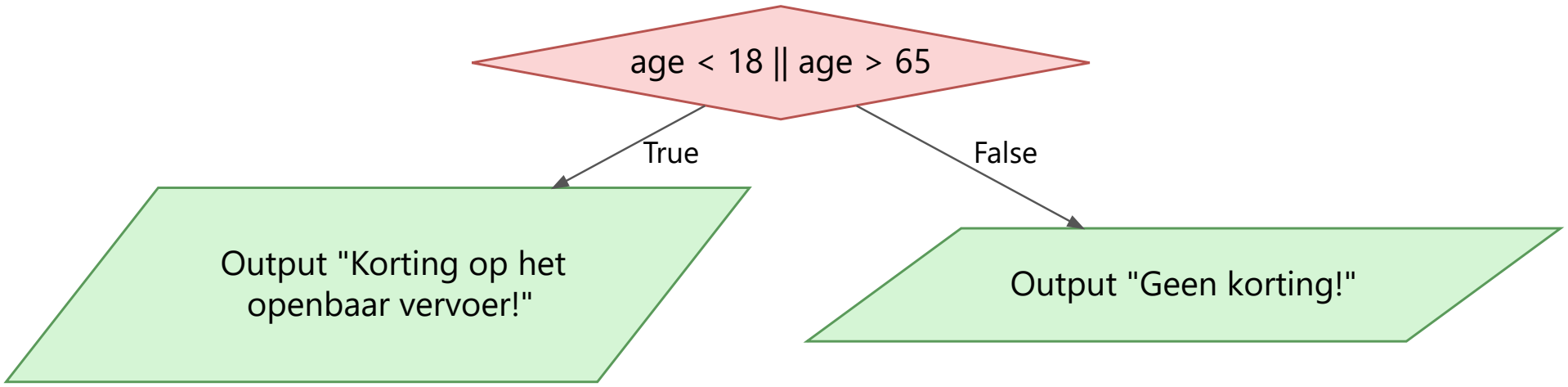
Twee ruiten, en **twee keer** dezelfde output.

🐞 *Twee keer dezelfde tekst typen? Copy-paste is geen programmeerstijl.*

Zonder `||` in C#

```
 1 if (age < 18)
 2 {
 3     Console.WriteLine("Korting op het openbaar vervoer!");
 4 }
 5 else if (age > 65)
 6 {
 7     Console.WriteLine("Korting op het openbaar vervoer!");
 8 }
 9 else
10 {
11     Console.WriteLine("Geen korting!");
12 }
```

Met ||




```
1  if (age < 18 || age > 65)
2  {
3      Console.WriteLine("Korting op het openbaar vervoer!");
4  }
5  else
6  {
7      Console.WriteLine("Geen korting!");
8  }
```

Ook met strings

```
1 Console.WriteLine("Welke dag is het?");
2 string day = Console.ReadLine();
3 if (day == "zaterdag" || day == "zondag")
4 {
5     Console.WriteLine("Weekend!");
6 }
```

> ⚠ **WAARSCHUWING**
>
> Elke kant van `||` moet een **volledige** vergelijking zijn. `day == "zaterdag" || "zondag"` compileert niet.

Weekend detecteren met `||`. Het nuttigste programma van het jaar.

Valkuilen

Intervallen

Is iemand **tussen 12 en 18 jaar** (grenzen inbegrepen)?

```
1  bool isTeenager = 12 <= age <= 18;      // ❌ CS0019
2  bool isTeenager = age >= 12 && <= 18;   // ❌ CS1525
```

```
1  bool isTeenager = age >= 12 && age <= 18;  // ✅
```

Twee **volledige** vergelijkingen, verbonden met `&&`.

 `12 <= age <= 18` *werkt in de wiskunde. C# heeft nooit wiskunde gevolgd.*

`&&` gaat voor `||`

```
1  Console.WriteLine(true || false && false);    // True
2  Console.WriteLine((true || false) && false);  // False
```

Net zoals `*` voor `+` gaat, wordt `&&` **eerst** berekend.

> **TIP**
>
> Twijfel je? Zet **haakjes**. Ze maken je bedoeling duidelijk, voor de compiler en voor de lezer.

Haakjes kosten niets. Je collega's bedanken je ermee.

Wat is de output? 🤔

```
1  if (temperature > 20 && !isRaining)
2  {
3      Console.WriteLine("Terras!");
4  }
5  else if (isRaining)
6  {
7      Console.WriteLine("Paraplu mee");
8  }
9  else
10 {
11     Console.WriteLine("Jas aan");
12 }
```

`temperature`	`isRaining`	**Output?**
25	`false`	`Terras!`
25	`true`	`Paraplu mee`
12	`false`	`Jas aan`

😈 *Terras bij 25 graden. In België heet dat meteen een hittegolf.*

Scope

Wat is scope?

De plaats waar je een variabele **declareert**, bepaalt waar je ze mag **gebruiken**.

Vuistregel: een variabele bestaat **vanaf** haar declaratie **tot** de sluitende accolade `}` van het blok waarin ze staat.

`if`, en straks ook loops en methoden, maken telkens een **nieuw blok** met accolades.

😈 *Wat in de accolades gebeurt, blijft in de accolades. Zoals Las Vegas, maar met meer puntkomma's.*

Te laat gebruikt

```
1  if (age >= 18)
2  {
3      string message = "Welkom!";
4  }
5  Console.WriteLine(message);  // ❌ CS0103
```

> ⚠ **WAARSCHUWING**
>
> The name ‘message’ does not exist in the current context

`message` bestaat enkel **binnen** de accolades van de `if`.

 *`message` is met zijn accolade mee naar huis.*

Oplossing: vroeger declareren

```
1  string message = "Te jong.";
2  if (age >= 18)
3  {
4      message = "Welkom!";
5  }
6  Console.WriteLine(message);  // ✅
```

Declareer de variabele **vóór** de `if`. Binnen de `if` geef je ze enkel een nieuwe waarde.

> **TIP**
>
> Geef ze meteen een **beginwaarde**. Zonder beginwaarde krijg je fout CS0165: C# kan niet garanderen dat de variabele altijd een waarde heeft.

Een variabele zonder waarde is als een lege brooddoos: C# weigert die mee te nemen.

Dezelfde naam

```
1  int number = 0;
2  if (number == 0)
3  {
4      int number = 5;  // ❌ CS0136
5  }
```

Zolang `number` in scope is, mag je **geen tweede** `number` maken.

Dit mag wel, want de eerste scope is al afgesloten:

```
1  {
2      int number = 0;
3  }
4  {
5      int number = 5;  // ✅
6  }
```

😈 *Twee variabelen met dezelfde naam in dezelfde scope? Zelfs C# raakt dan de tel kwijt.*

Samengevat

Wat	Hoe
allebei waar	`a && b`
minstens één waar	`a || b`
omdraaien	`!a`
interval	`age >= 12 && age <= 18`
scope	van de declaratie tot de `}` van het blok

BELANGRIJK

Elke kant van `&&` en `||` is een **volledige** vergelijking. Bij twijfel: **haakjes**.

`&&`, `||` en `!`: drie tekens, oneindig veel manieren om een voorwaarde fout te schrijven.

Aan de slag 💻

Maak verder in de klasse `Beslissingen`:

1. **ShoeSeller** (uitbreiding): korting enkel met een klantenkaart
2. **LargestNumberOfThree**: bepaal het grootste van drie getallen
3. **Exams**: geslaagd met een gemiddelde boven 50% en maximaal 1 onvoldoende
4. **LeapYear**: is een jaartal een schrikkeljaar?

👉 Labo 1 nog niet af? Werk dan eerst EvenOrOdd, PositiveNegativeZero, BMICalculator, OhmsLaw en PocketMoney af.

😈 `if (understood || !understood) { /* oefeningen maken */ }` *is altijd* `true`. *Geen ontsnappen aan.*